Практическая работа №6

«Разработка многопроцессной системы анализа данных на Python»

Выполнил: Ламин Дахмуни

Группа: 1-КПД-2

1. Цель работы

Написать программу на Python, которая анализирует массив из 1 000 000 случайных чисел двумя способами — последовательно и с помощью нескольких процессов (модуль **multiprocessing**), — сравнить результаты и время работы, а также сделать консольное меню для управления программой.

2. Как устроена программа

Этап I. Данные генерируются функцией **generate_data(n)** с помощью **random**. Перед генерацией вызывается **random.seed(42)**, поэтому при каждом запуске получается один и тот же массив.

Этап II. Для каждой операции написана отдельная функция: **calculate_sum**, **calculate_average**, **find_min_max**, **count_even_odd**, **count_divisible**, **find_top_10**, **sort_data**. Все функции возвращают результат и не изменяют исходный массив (сортировка делается через **sorted()**, который создаёт новый список). Результаты складываются в словарь **result**, общее время измеряется через **time.perf_counter()**.

Этап III. Функция **split_data(data, k)** делит массив на k частей. Каждая часть отправляется в свой процесс через **Pool.map**. Функция **worker(part)** в процессе считает: количество элементов, сумму, минимум, максимум, чётные/нечётные, делимость на 3, 5 и 7, счётчик **Counter** и отсортированную часть, после чего возвращает всё это словарём.

Этап IV. Объединение результатов (функция **merge_results**):

• **сумма, количество чётных/нечётных, количество делящихся на 3, 5, 7** — просто складываются по всем частям;

• **минимум и максимум** — берётся минимум из минимумов и максимум из максимумов;

• **среднее арифметическое** — считается как (общая сумма) / (общее количество элементов). Четыре средних нельзя просто усреднять, потому что части могут быть разного размера;

• **топ-10** — счётчики Counter всех частей складываются в один, затем выбираются 10 самых частых чисел (при равной частоте раньше идёт меньшее число, чтобы результат совпадал с последовательным вариантом);

• **сортировка** — отсортированные части сливаются в один массив алгоритмом слияния (**heapq.merge**), повторно сортировать весь массив не нужно.

Если размер массива не делится на число процессов, функция **split_data** использует **divmod(n, k)**: первые (n mod k) частей получают по одному лишнему элементу. Например, 1 000 003 числа на 8 процессов делятся на части 125 001, 125 001, 125 001, 125 000, … Проверка показала, что результаты и в этом случае совпадают с последовательными.

3. Текст программы (practice6.py)

```
# Практическая работа №6
# «Разработка многопроцессной системы анализа данных на Python»
# Выполнил: Ламин Дахмуни, группа 1-КПД-2

import random
import time
import heapq
from collections import Counter
from multiprocessing import Pool

SEED = 42  # фиксированное начальное значение генератора


# ---------------- Этап I. Генерация данных ----------------
def generate_data(n):
    random.seed(SEED)
    return [random.randint(1, 10000) for _ in range(n)]


# ---------------- Этап II. Отдельные функции анализа ----------------
def calculate_sum(data):
    return sum(data)


def calculate_average(data):
    return sum(data) / len(data)


def find_min_max(data):
    return min(data), max(data)


def count_even_odd(data):
    even = 0
    for x in data:
        if x % 2 == 0:
            even += 1
    return even, len(data) - even


def count_divisible(data):
    d3 = d5 = d7 = 0
    for x in data:
        if x % 3 == 0:
            d3 += 1
        if x % 5 == 0:
            d5 += 1
        if x % 7 == 0:
            d7 += 1
    return d3, d5, d7


def top_from_counter(counter):
    # при равной частоте меньшее число идёт первым (чтобы результат был однозначным)
    items = sorted(counter.items(), key=lambda p: (-p[1], p[0]))
    return items[:10]


def find_top_10(data):
    return top_from_counter(Counter(data))


def sort_data(data):
    return sorted(data)  # исходный массив не меняется


# ---------------- Последовательный анализ ----------------
def sequential_analysis(data):
    start = time.perf_counter()
    result = {}
    result["sum"] = calculate_sum(data)
    result["average"] = calculate_average(data)
    result["min"], result["max"] = find_min_max(data)
    result["even"], result["odd"] = count_even_odd(data)
    result["div3"], result["div5"], result["div7"] = count_divisible(data)
```

```
    result["top10"] = find_top_10(data)
    result["sorted"] = sort_data(data)
    result["time"] = time.perf_counter() - start
    return result


# ---------------- Этап III. Работа одного процесса ----------------
def worker(part):
    """Анализ одной части массива. Возвращает частичные результаты."""
    mn, mx = find_min_max(part)
    even, odd = count_even_odd(part)
    d3, d5, d7 = count_divisible(part)
    return {
        "count": len(part),
        "sum": calculate_sum(part),
        "min": mn,
        "max": mx,
        "even": even,
        "odd": odd,
        "div3": d3,
        "div5": d5,
        "div7": d7,
        "counter": Counter(part),
        "sorted": sort_data(part),
    }


def split_data(data, k):
    """Делит массив на k частей. Если размер не делится нацело,
    первые (n % k) частей получают по одному лишнему элементу."""
    n = len(data)
    base, extra = divmod(n, k)
    parts = []
    pos = 0
    for i in range(k):
        size = base + (1 if i < extra else 0)
        parts.append(data[pos:pos + size])
        pos += size
    return parts


# ---------------- Этап IV. Объединение результатов ----------------
def merge_results(partials):
    total_count = sum(p["count"] for p in partials)
    total_sum = sum(p["sum"] for p in partials)
    counter = Counter()
    for p in partials:
        counter.update(p["counter"])
    result = {
        "sum": total_sum,
        # среднее считаем через общую сумму и общее количество элементов
        "average": total_sum / total_count,
        "min": min(p["min"] for p in partials),
        "max": max(p["max"] for p in partials),
        "even": sum(p["even"] for p in partials),
        "odd": sum(p["odd"] for p in partials),
        "div3": sum(p["div3"] for p in partials),
        "div5": sum(p["div5"] for p in partials),
        "div7": sum(p["div7"] for p in partials),
        "top10": top_from_counter(counter),
        # слияние уже отсортированных частей
        "sorted": list(heapq.merge(*[p["sorted"] for p in partials])),
    }
    return result


def parallel_analysis(data, processes):
    start = time.perf_counter()
    parts = split_data(data, processes)
    with Pool(processes) as pool:
        partials = pool.map(worker, parts)
    result = merge_results(partials)
    result["time"] = time.perf_counter() - start
    return result


# ---------------- Сравнение результатов ----------------
KEYS = ["sum", "average", "min", "max", "even", "odd",
```

```
        "div3", "div5", "div7", "top10", "sorted"]


def results_equal(a, b):
    for key in KEYS:
        if a[key] != b[key]:
            return False
    return True


# ---------------- Вывод ----------------
def print_stats(res):
    print("  Сумма:                 ", res["sum"])
    print("  Среднее арифметическое:", round(res["average"], 4))
    print("  Минимум / максимум:    ", res["min"], "/", res["max"])
    print("  Чётных / нечётных:     ", res["even"], "/", res["odd"])
    print("  Делятся на 3 / 5 / 7:  ", res["div3"], "/", res["div5"], "/", res["div7"])
    print("  Топ-10 частых чисел (число: сколько раз):")
    print("   ", ", ".join(f"{v}: {c}" for v, c in res["top10"]))
    s = res["sorted"]
    print("  Первые 5 после сортировки:", s[:5])
    print("  Последние 5 после сортировки:", s[-5:])
    print("  Время выполнения: %.4f сек" % res["time"])


# ---------------- Ввод с проверкой ----------------
def input_int(text, min_value, max_value):
    while True:
        s = input(text).strip()
        try:
            value = int(s)
        except ValueError:
            print("Ошибка: нужно ввести целое число.")
            continue
        if value < min_value or value > max_value:
            print(f"Ошибка: число должно быть от {min_value} до {max_value}.")
            continue
        return value


def show_menu():
    print("==================================")
    print("     СИСТЕМА АНАЛИЗА ДАННЫХ")
    print("==================================")
    print("1. Сгенерировать данные")
    print("2. Последовательный анализ")
    print("3. Параллельный анализ")
    print("4. Сравнить производительность")
    print("5. Показать статистику")
    print("6. Выход")


def main():
    data = None
    seq_res = None
    par_res = None
    last_processes = None

    while True:
        show_menu()
        try:
            choice = input("Выберите действие: ").strip()
        except EOFError:
            print("\nВвод закончен. Выход.")
            break

        if choice == "1":
            n = input_int("Сколько чисел сгенерировать (1 - 5000000)? ", 1, 5000000)
            t = time.perf_counter()
            data = generate_data(n)
            seq_res = par_res = None
            print(f"Сгенерировано {n} чисел за {time.perf_counter() - t:.4f} сек.")

        elif choice == "2":
            if data is None:
                print("Сначала сгенерируйте данные (пункт 1).")
            else:
                seq_res = sequential_analysis(data)
```

```
            print("Последовательный анализ завершён.")
            print_stats(seq_res)

        elif choice == "3":
            if data is None:
                print("Сначала сгенерируйте данные (пункт 1).")
            else:
                k = input_int("Сколько процессов использовать (1 - 16)? ", 1, 16)
                if k > len(data):
                    print("Процессов не может быть больше, чем чисел.")
                else:
                    last_processes = k
                    par_res = parallel_analysis(data, k)
                    print(f"Параллельный анализ ({k} проц.) завершён.")
                    print_stats(par_res)

        elif choice == "4":
            if data is None:
                print("Сначала сгенерируйте данные (пункт 1).")
            else:
                k = input_int("Сколько процессов использовать (1 - 16)? ", 1, 16)
                if k > len(data):
                    print("Процессов не может быть больше, чем чисел.")
                else:
                    last_processes = k
                    seq_res = sequential_analysis(data)
                    par_res = parallel_analysis(data, k)
                    print("Последовательно: %.4f сек" % seq_res["time"])
                    print("Параллельно (%d проц.): %.4f сек" % (k, par_res["time"]))
                    print("Ускорение S = T1/Tn = %.2f" % (seq_res["time"] / par_res["time"]))
                    if results_equal(seq_res, par_res):
                        print("Результаты последовательного и параллельного анализа СОВПАДАЮТ.")
                    else:
                        print("Результаты НЕ совпадают!")

        elif choice == "5":
            if seq_res is None and par_res is None:
                print("Статистики пока нет. Выполните анализ (пункт 2 или 3).")
            else:
                if seq_res is not None:
                    print("--- Последовательный анализ ---")
                    print_stats(seq_res)
                if par_res is not None:
                    print(f"--- Параллельный анализ ({last_processes} проц.) ---")
                    print_stats(par_res)

        elif choice == "6":
            print("До свидания!")
            break

        else:
            print("Ошибка: выберите пункт меню от 1 до 6.")
        print()


if __name__ == "__main__":
    main()
```

4. Работа консольного интерфейса

Ниже показан запуск программы: сначала неверный ввод (программа не завершается аварийно, а просит повторить), затем генерация 1 000 000 чисел, последовательный и параллельный анализ и сравнение.

```
================================
     СИСТЕМА АНАЛИЗА ДАННЫХ
================================
1. Сгенерировать данные
2. Последовательный анализ
3. Параллельный анализ
4. Сравнить производительность
5. Показать статистику
6. Выход
Выберите действие: abc
Ошибка: выберите пункт меню от 1 до 6.
Выберите действие: 2
Сначала сгенерируйте данные (пункт 1).
Выберите действие: 1
Сколько чисел сгенерировать (1 - 5000000)? абв
Ошибка: нужно ввести целое число.
Сколько чисел сгенерировать (1 - 5000000)? -5
Ошибка: число должно быть от 1 до 5000000.
Сколько чисел сгенерировать (1 - 5000000)? 1000000
Сгенерировано 1000000 чисел за 0.4171 сек.

Выберите действие: 2
Последовательный анализ завершён.
  Сумма:                   5000414737
  Среднее арифметическое: 5000.4147
  Минимум / максимум:      1 / 10000
  Чётных / нечётных:       500537 / 499463
  Делятся на 3 / 5 / 7:    332905 / 200358 / 142220
  Топ-10 частых чисел (число: сколько раз):
    7506: 141, 3592: 138, 5980: 136, 232: 135, 3384: 134,
    898: 133, 1668: 133, 5499: 133, 6282: 133, 2953: 132
  Первые 5 после сортировки: [1, 1, 1, 1, 1]
  Последние 5 после сортировки: [10000, 10000, 10000, 10000, 10000]
  Время выполнения: 0.4464 сек

Выберите действие: 3
Сколько процессов использовать (1 - 16)? 0
Ошибка: число должно быть от 1 до 16.
Сколько процессов использовать (1 - 16)? 4
Параллельный анализ (4 проц.) завершён.
  (те же значения, что и при последовательном анализе)
  Время выполнения: 0.8495 сек

Выберите действие: 4
Сколько процессов использовать (1 - 16)? 4
Последовательно: 0.4301 сек
Параллельно (4 проц.): 0.8495 сек
Ускорение S = T1/Tn = 0.51
Результаты последовательного и параллельного анализа СОВПАДАЮТ.

Выберите действие: 6
До свидания!
```

5. Эксперимент с производительностью

Массив — 1 000 000 чисел (seed = 42). Для каждого количества процессов программа запускалась 3 раза, в таблице записано лучшее время. Во всех запусках результаты параллельного анализа совпали с последовательным. Ускорение считается по формуле S = T1 / Tn, где T1 — время работы с 1 процессом, Tn — время работы с n процессами.

Количество процессов	Время выполнения, с	Ускорение S = T1/Tn	Ускорение относительно последовательной версии
1	0.865	1.00	0.59
2	1.051	0.82	0.48
4	0.854	1.01	0.59
8	0.974	0.89	0.52

Время чисто последовательной программы (без процессов): **0.508 с**.

Вывод по эксперименту. Тесты выполнялись на компьютере с **одним ядром процессора**, поэтому реальных параллельных вычислений не было: все процессы по очереди делили одно ядро. В результате ускорения нет, а параллельная версия даже медленнее последовательной (0.865 с против 0.508 с), потому что к вычислениям добавились затраты на создание процессов, пересылку частей массива туда и обратно и объединение результатов. На компьютере с 4 и более ядрами при тех же программе и данных ожидается заметное ускорение у версий с 2 и 4 процессами.

6. Анализ эффективности (ответы на вопросы)

1. Почему параллельная программа не всегда работает быстрее последовательной?

Потому что у параллельности есть накладные расходы: время на создание процессов, на передачу данных между ними, на сбор и объединение результатов. Если сама работа небольшая или ядро всего одно, эти расходы больше, чем выигрыш — как это и получилось в моём эксперименте.

2. Как влияет количество процессов на скорость вычислений?

Пока число процессов не больше числа ядер процессора, скорость растёт (почти пропорционально). Когда процессов становится больше, чем ядер, ускорение прекращается: процессы ждут своей очереди, а расходы на их создание и переключение только растут. Поэтому 8 процессов на 4 ядрах не быстрее 4 процессов.

3. Почему передача больших массивов между процессами может замедлять программу?

Процессы не имеют общей памяти. Массив нужно сериализовать (pickle), скопировать в другой процесс и там восстановить, а результат так же вернуть обратно. Для миллиона чисел это занимает время, сравнимое с самим подсчётом, особенно для простых операций вроде суммы.

4. Какие вычисления проще всего распределить между процессами?

Те, в которых части данных обрабатываются независимо, а итог получается простым объединением: сумма, количество чётных, минимум/максимум, подсчёт делимости. Каждый процесс считает свою часть, а главная программа складывает числа.

5. Почему сортировку сложнее распараллелить, чем подсчёт чётных чисел?

Подсчёт чётных не требует связи между элементами — части совершенно независимы. При сортировке порядок каждого элемента зависит от всего массива: после сортировки частей их ещё нужно слить в один массив, а это последовательная операция по всем данным, которая плохо делится между процессами. К тому же отсортированные части приходится пересылать обратно целиком.

6. Что произойдёт, если один процесс завершится с ошибкой?

При использовании Pool.map исключение из процесса передаётся в главную программу и вызов pool.map завершается ошибкой, а результаты остальных процессов теряются — полный результат собрать нельзя. Чтобы программа не падала, вызов нужно обернуть в try/except и, например, повторить обработку этой части (в другом процессе или последовательно) либо сообщить пользователю об ошибке.

7. Как организовать обработку массива, если его размер не делится на количество процессов?

Нужно разделить массив с остатком: если n = k·base + extra, то первые extra частей получают base+1 элементов, остальные — base. Именно так работает моя функция split_data (через divmod). Размеры частей отличаются максимум на 1, и поэтому нагрузка распределяется почти поровну. При объединении среднее нужно считать по общему количеству элементов, а не как среднее средних.

7. Общий вывод

В работе реализованы последовательный и параллельный (на основе multiprocessing) анализ массива из 1 000 000 чисел, правильное объединение результатов (в том числе среднего, топ-10 и сортировки), измерено время при 1, 2, 4 и 8 процессах и создан консольный интерфейс с проверкой ввода. Результаты обоих вариантов полностью совпадают. Эксперимент показал, что параллельная обработка даёт выигрыш только тогда, когда есть несколько ядер и объём вычислений больше накладных расходов на передачу данных.